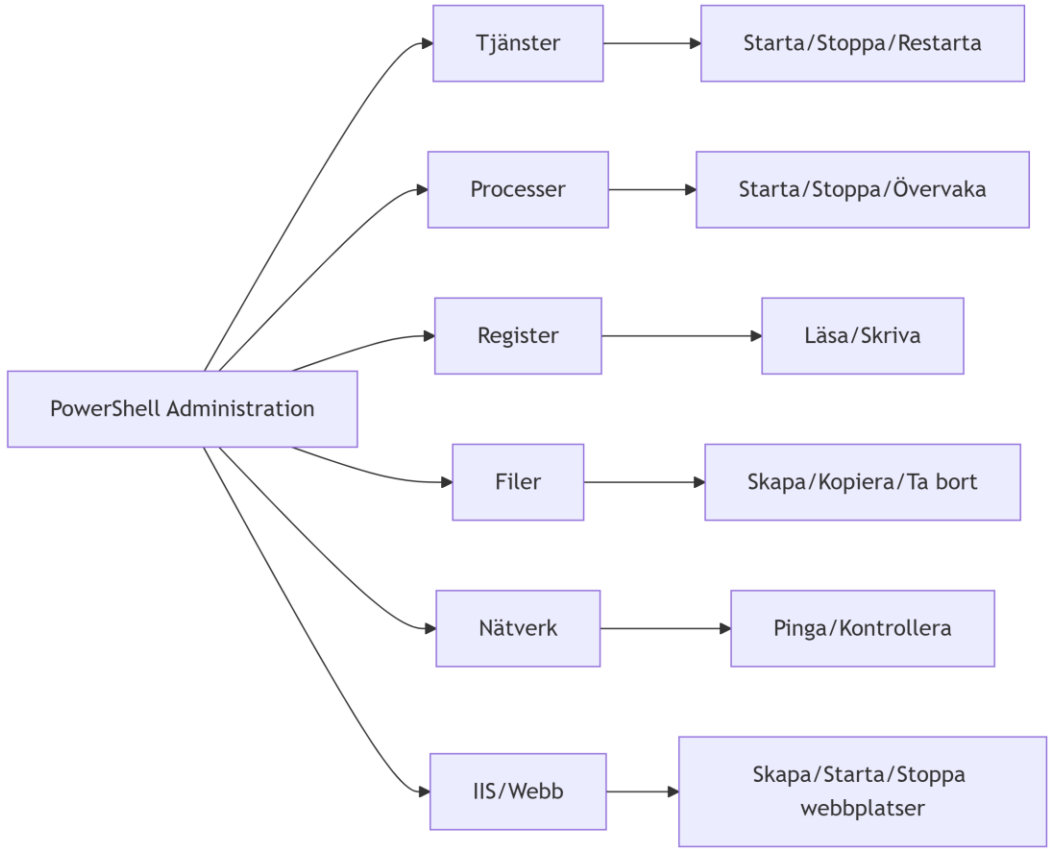
PowerShell Systemadministration

Arseniy Buzyan

En Introduktion

PowerShell är standardverktyget för att administrera Windows-system i Sverige.

Varför? PowerShell kan hantera ALLA aspekter av Windows – tjänster, processer, register, filer, nätverk, Active Directory och IIS .



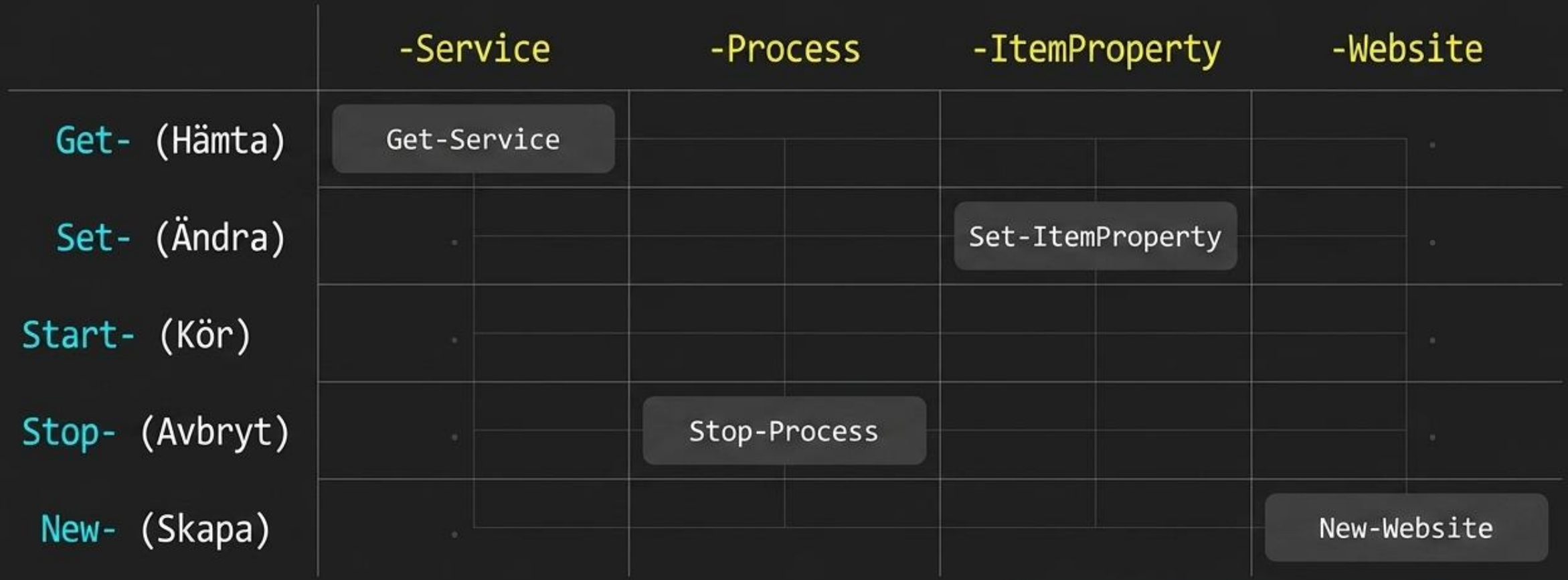
Microsoft.PowerShell.Management-modulen innehåller över 70 inbyggda cmdlets för systemadministration .

Microsoft.PowerShell.Management – Ditt Verktygslåda

Alla systemadministrations-cmdlets finns i modulen Microsoft.PowerShell.Management.

Så här ser du alla kommandon:
Get-Command -Module Microsoft.PowerShell.Management

Grammatiken: Verb-Substantiv



PowerShell är förutsägbart. Lär dig verben, så kan du gissa kommandot.

De viktigaste cmdlets för dig

Cmdlet	Vad gör den?	Exempel
Get-Service	Visa alla tjänster	Get-Service
Start-Service	Starta en tjänst	Start-Service -Name W3SVC
Stop-Service	Stoppa en tjänst	Stop-Service -Name Spooler
Restart-Service	Starta om en tjänst	Restart-Service -Name W3SVC
Get-Process	Visa alla processer	Get-Process
Stop-Process	Stoppa en process	Stop-Process -Name notepad
Get-ItemProperty	Läs register-värde	Get-ItemProperty -Path HKLM:\SOFTWARE\...
Set-ItemProperty	Ändra register-värde	Set-ItemProperty -Path ... -Name "Version" -Value "1.0"

Diagnostik: Tjänster vs. Processer



Tjänster (Services)

Definition: Program som körs i bakgrunden (t.ex. webbserver, nätverk).

Nyckelcmdlets:

```
Get-Service
Restart-Service
```

Exempel på felsökning: En webbsida laddar inte? Starta om W3SVC-tjänsten.



Processer (Processes)

Definition: Aktiva program som körs på datorn för tillfället.

Nyckelcmdlets:

```
Get-Process
Stop-Process
```

Exempel på felsökning: En applikation har hängt sig och slukar CPU? Tvinga fram ett stopp på processen.

Hantera Tjänster – Grunden i Windows



Tjänster = Program som körs i bakgrunden (t.ex. webbserver, skrivare, nätverk).



Viktiga kommandon:

Cmdlet	Vad gör den?	Exempel
Get-Service	Visa alla tjänster	Get-Service
Get-Service -Name	Visa en specifik tjänst	Get-Service -Name W3SVC
Start-Service	Starta en tjänst	Start-Service -Name W3SVC
Stop-Service	Stoppa en tjänst	Stop-Service -Name Spooler
Restart-Service	Starta om en tjänst	Restart-Service -Name W3SVC

Exempel – filtrera tjänster:

- Exempel – filtrera tjänster:
- powershell

```powershell
# Visa alla tjänster som är igång
Get-Service | Where-Object {$_.Status -eq "Running"}

# Visa alla tjänster som börjar med "win"
Get-Service -Name "win*"
```

- Exempel – starta om en tjänst om den är stoppad:
- powershell

```powershell
$service = Get-Service -Name "Spooler"
if ($service.Status -eq "Stopped") {
    Start-Service -Name "Spooler"
    Write-Host "Spooler startades!"
}
```

Hantera Processer – Program som Körs

Processer = Aktiva program som körs på din dator

Cmdlet	Vad gör den?	Exempel
Get-Process	Visa alla processer	Get-Process
Get-Process -Name	Visa en specifik process	Get-Process -Name notepad
Start-Process	Starta en process	Start-Process -FilePath "C:\Windows\notepad.exe"
Stop-Process	Stoppa en process	Stop-Process -Name notepad

Exempel – hantera processer med pipeline

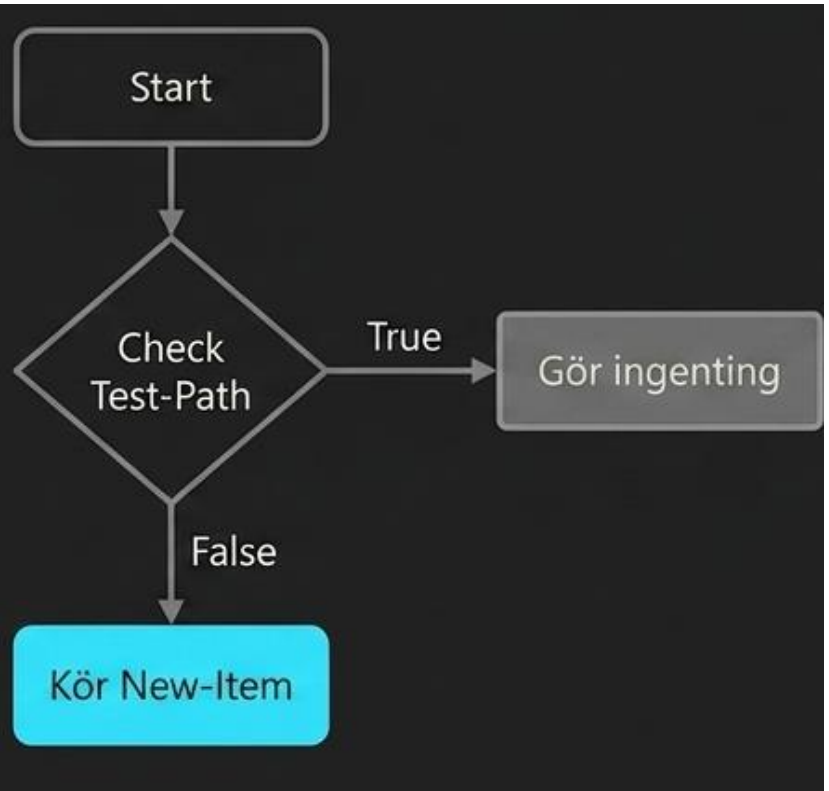
- **Proffstips:** Använd `Get-Process | Get-Member` för att se alla egenskaper hos en process.
- powershell
- `Get-Process -Name notepad | Stop-Process`
-
- `# Visa de 5 processer som använder mest CPU`
- `Get-Process | Sort-Object CPU -Descending | Select-Object -First 5`
-
- `# Stoppa en process om den använder för mycket CPU`
- `$process = Get-Process -Name "chrome" | Sort-Object CPU -Descending | Select-Object -First 1`
- `if ($process.CPU -gt 50) {`
- `    Stop-Process -Name "chrome"`
- `    Write-Host "Chrome stoppades (för hög CPU-användning)"`
- `}`

Grunden i ALL systemadministration – skapa, kopiera, ta bort och flytta filer.

Hantera Filer och Mappar

Cmdlet	Vad gör den?	Exempel
New-Item	Skapa fil eller mapp	New-Item -Path "C:\temp" -ItemType Directory
Copy-Item	Kopiera fil/mapp	Copy-Item -Path "C:\source\file.txt" -Destination "C:\dest\"
Move-Item	Flytta fil/mapp	Move-Item -Path "C:\source\file.txt" -Destination "C:\dest\"
Remove-Item	Ta bort fil/mapp	Remove-Item -Path "C:\temp\file.txt"
Get-ChildItem	Lista filer/mappar	Get-ChildItem -Path "C:\temp"
Test-Path	Kontrollera om fil/mapp finns	Test-Path -Path "C:\temp"

Exempel – skapa mappstruktur automatiskt:



- powershell
- `# Skapa en hel mappstruktur`
- `$mappar = @("C:\Projekt", "C:\Projekt\Logs", "C:\Projekt\Config")`
- `foreach ($mapp in $mappar) {`
- `    if (-not (Test-Path $mapp)) {`
- `        New-Item -Path $mapp -ItemType Directory`
- `        Write-Host "Skapade: $mapp"`
- `    }`
- `}`
- **Proffstips:** `Test-Path` är ovärderlig – den kontrollerar om en fil/mapp finns INNAN du försöker använda den.

Hantera Registret – Windows Hjärta



Registret = Windows "hjärna" – alla inställningar lagras här.



PowerShell-registerprovidern ger dig tillgång till registret som en enhet (HKLM:, HKCU:) .

Cmdlet	Vad gör den?	Exempel
Get-ItemProperty	Läs register-värde	Get-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion"
Set-ItemProperty	Ändra register-värde	Set-ItemProperty -Path "HKLM:\SOFTWARE\MyApp" -Name "Version" -Value "1.0"
New-Item	Skapa nyckel	New-Item -Path "HKLM:\SOFTWARE\MyApp"
Get-ItemPropertyValue	Hämta ENDAST värdet	Get-ItemPropertyValue -Path "HKLM:\SOFTWARE\MyApp" -Name "Version"

Exempel – läsa register-värde

```powershell
# Visa vilka program som startar automatiskt
Get-ChildItem -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run"

# Läs ett specifikt värde
$path = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion"
Get-ItemProperty -Path $path -Name "ProgramFilesDir"

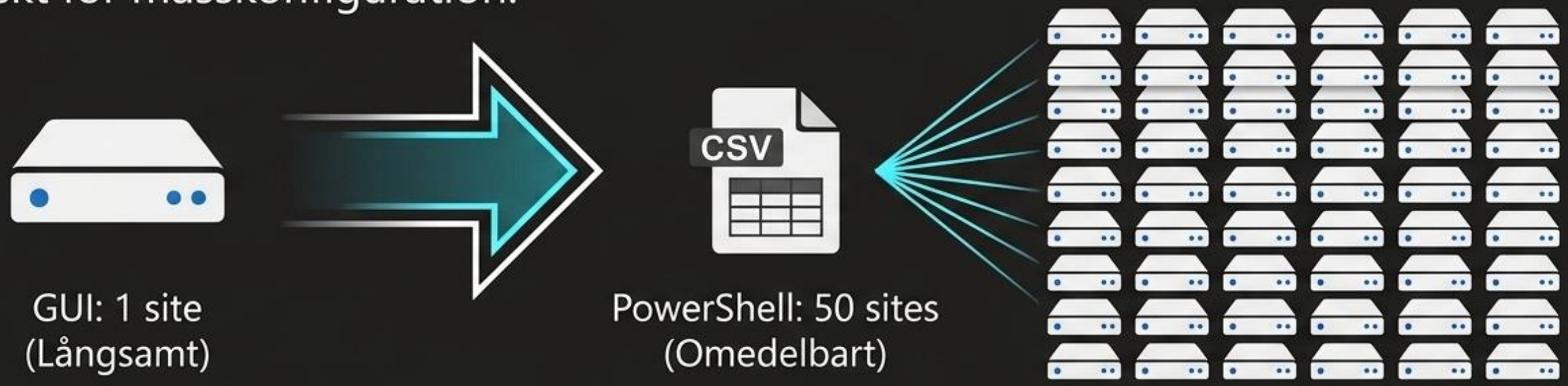
# I PowerShell 5.0+ kan du använda Get-ItemPropertyValue för att bara få värdet
Get-ItemPropertyValue -Path $path -Name "ProgramFilesDir"
```

VARNING: Var försiktig med registret – fel ändringar kan göra datorn instabil!

```powershell
# Ändra starttyp på en tjänst via registret
$path = "HKLM:\SYSTEM\CurrentControlSet\Services\Spooler\"
Set-ItemProperty -Path $path -Name "Start" -Value 3  # 3 = Manual
# Bättre: använd Set-Service istället!
Set-Service -Name Spooler -StartupType Manual
```

Webb-Administration (IIS): Skala Upp

Modulen WebAdministration exponerar IIS som en navigerbar enhet (IIS:\).
Perfekt för masskonfiguration.

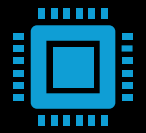



```powershell
Import-Module WebAdministration
foreach ($s in $sites) {
    New-WebAppPool -Name $s.AppPool
    New-Website -Name $s.Name -PhysicalPath $s.PhysicalPath -ApplicationPool $s.AppPool
}
```

Hantera IIS med PowerShell (Webbadministration)



PowerShell kan hantera hela IIS (webbserver) – perfekt för automatisering!



Modulen WebAdministration levereras med Windows Server och innehåller IIS-cmdlets .

Viktiga IIS-cmdlets

Cmdlet	Vad gör den?	Exempel
Import-Module WebAdministration	Ladda IIS-modulen	Import-Module WebAdministration
Get-Website	Visa alla webbplatser	Get-Website
New-Website	Skapa webbplats	New-Website -Name "ContosoSite" -PhysicalPath "C:\inetpub\ContosoSite" -Port 80
Start-Website	Starta webbplats	Start-Website -Name "ContosoSite"
Stop-Website	Stoppa webbplats	Stop-Website -Name "ContosoSite"
Remove-Website	Ta bort webbplats	Remove-Website -Name "ContosoSite"
New-WebAppPool	Skapa applikationspool	New-WebAppPool -Name "ContosoPool"

Exempel – skapa en webbplats med PowerShell

- Proffstips: IIS-modulen exponerar IIS-konfigurationen som en navigerbar enhet (IIS:\) – precis som registret !

```powershell
# Ladda IIS-modulen
Import-Module WebAdministration

# Skapa en applikationspool
New-WebAppPool -Name "ContosoPool"

# Skapa webbplatsen
New-Website -Name "ContosoSite" `
    -PhysicalPath "C:\inetpub\ContosoSite" `
    -Port 80 `
    -ApplicationPool "ContosoPool"

# Starta webbplatsen
Start-Website -Name "ContosoSite"
```

Masskonfiguration med PowerShell – Verkliga Exempel

En vanlig produktionsuppgift: tillämpa samma inställning på alla webbplatser .

Exempel – inaktivera katalogbläddring på ALLA webbplatser:

```
Import-Module WebAdministration
foreach ($site in Get-Website) {
    Set-WebConfigurationProperty `
        -Filter
"/system.webServer/directoryBrowse" `
        -PSPath "IIS:\Sites\$($site.Name)" `
        -Name enabled -Value $false
    Write-Host "Directory browsing disabled:
$($site.Name)"
}
```



Exempel – skapa 50 webbplatser från en CSV-fil:

```
•
• foreach ($s in $sites) {
•     # Skapa applikationspool om den inte finns
•     if (-not (Test-Path "IIS:\AppPools\$($s.AppPool)")) {
•         New-WebAppPool -Name $s.AppPool
•     }
•
•     # Skapa mapp om den inte finns
•     if (-not (Test-Path $s.PhysicalPath)) {
•         New-Item -ItemType Directory -Path $s.PhysicalPath |
  Out-Null
•     }
•
•     # Skapa webbplats
•     New-Website -Name $s.Name `
•         -PhysicalPath $s.PhysicalPath `
•         -HostHeader $s.HostHeader `
•         -Port $s.Port `
•         -ApplicationPool $s.AppPool
```

Verktygsval: AppCmd.exe vs. PowerShell

AppCmd.exe

När?
Snabba ad-hoc frågor i äldre system, eller när PowerShell-moduler saknas.

Varför?
Snabbt, kräver inga moduler. Returnerar dock ren text.

PowerShell

När?
Standard för moderna miljöer och masskonfiguration.

Varför?
Objektorienterat. Kan enkelt skickas genom pipelinen till CSV, HTML, eller filter.

Testa Nätverksanslutning och Systeminformation

- Kontrollera om servrar är igång och hämta systeminformation.

Cmdlet	Vad gör den?	Exempel
Test-Connection	Pinga en server	Test-Connection -ComputerName "google.com" -Count 2
Get-ComputerInfo	Hämta systeminformation	Get-ComputerInfo
Get-NetIPAddress	Hämta IP-adresser	Get-NetIPAddress -AddressFamily IPv4

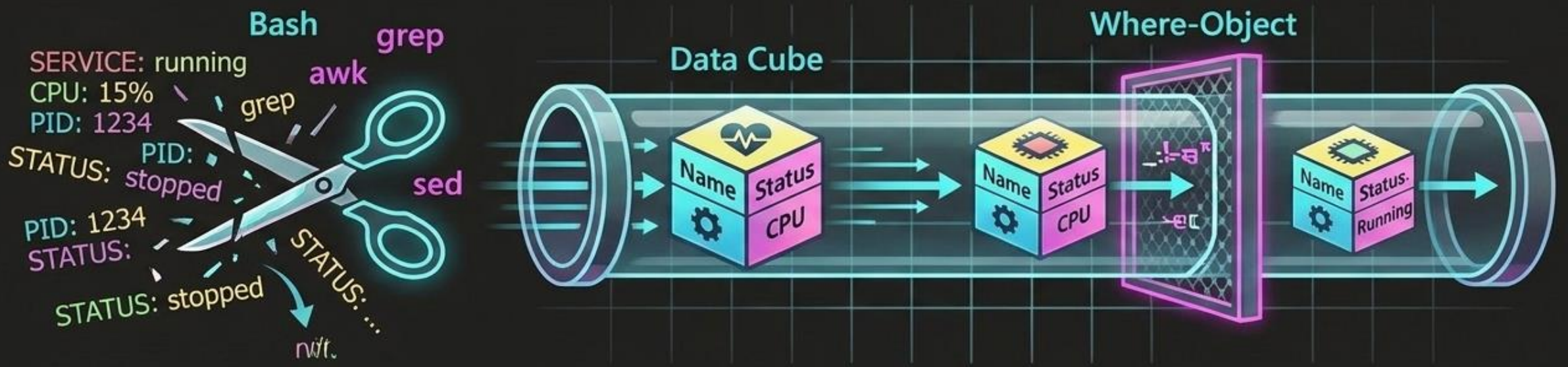
Exempel – kontrollera att en server är igång:

- Proffstips: Använd -Quiet för att få ett sant/falskt-svar istället för ping-statistik.

```
powershell
# Pinga en server och visa resultat
$server = "webserver01"
$resultat = Test-Connection -ComputerName $server -Count 2 -Quiet
if ($resultat) {
    Write-Host "$server svarar!" -ForegroundColor Green
} else {
```

Pipelinen (|): Objekt, Inte Text

I äldre skal (som Bash) måste du manipulera rå text med `awk` eller `sed`.
I PowerShell skickar pipelinen rika dataobjekt.



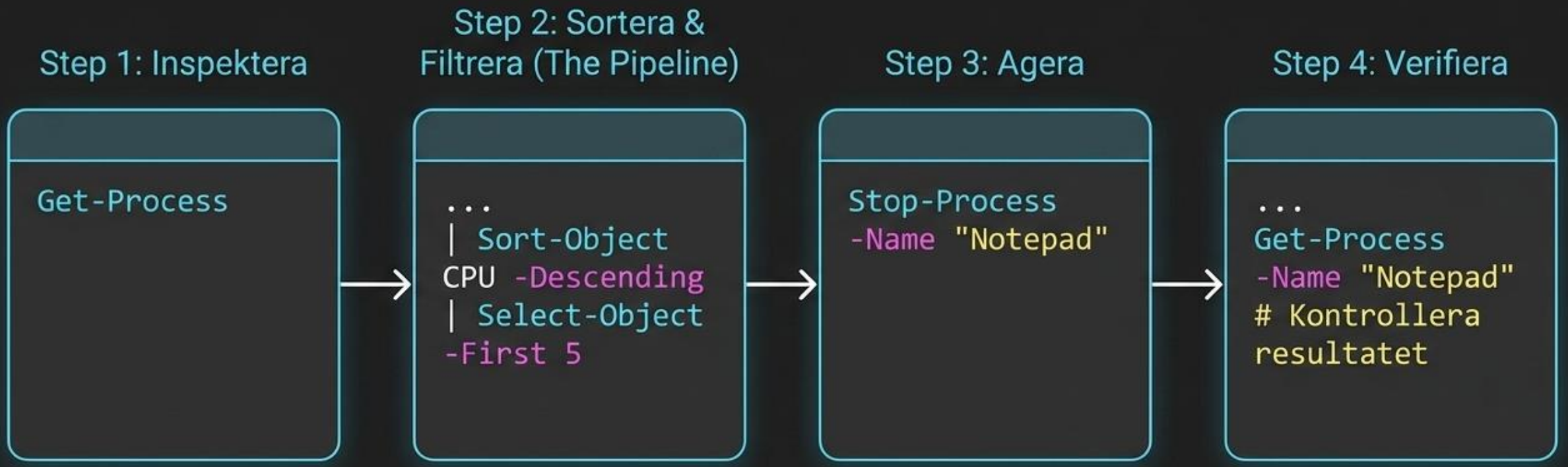

```powershell
# Hämta rika objekt -> Filtrera på egenskap -> Välj exakt data
Get-Service | Where-Object {$_.Status -eq "Running"} | Select-Object Name, Status
```

Exempel – kombinera kommandon

- **Varför detta är viktigt:** I Bash måste du använda `awk`, `sed` och `grep` för att tolka text. I PowerShell är allt redan objekt – du behöver bara välja ut de egenskaper du vill ha .

- powershell
- `# Hämta tjänster → filtrera på status → välj ut namn och status`
- `Get-Service | Where-Object {$_.Status -eq "Running"} | Select-Object Name, Status`
-
- `# Hämta processer → sortera efter CPU → visa de 3 högsta`
- `Get-Process | Sort-Object CPU -Descending | Select-Object -First 3 Name, CPU`

Sammanfattning: Automationsflödet



PowerShell är inte bara en lista med kommandon.
Det är ett enhetligt, objektorienterat system för total kontroll.



Workshop – Hantera Tjänster och Processer

Tänk: Detta är grunden till all PowerShell-automatisering – inspektera, filtrera, ändra, verifiera.

Mål: Använda PowerShell för att hantera tjänster och processer.

- **Steg 1:** Visa alla tjänster
- powershell
- `Get-Service`
- **Steg 2:** Visa alla tjänster som är igång
- powershell
- `Get-Service | Where-Object {$_.Status -eq "Running"}`
- **Steg 3:** Stoppa skrivartjänsten
- powershell
- `Stop-Service -Name "Spooler"`
- `Get-Service -Name "Spooler"  # Kontrollera att den är stoppad`
- **Steg 4:** Starta skrivartjänsten igen
- powershell
- `Start-Service -Name "Spooler"`
- **Steg 5:** Visa alla processer och hitta den som använder mest CPU
- powershell
- `Get-Process | Sort-Object CPU -Descending | Select-Object -First 5`
- **Steg 6:** Avsluta en process (t.ex. Notepad)
- powershell
- `# Starta Notepad först (om den inte är igång)`
- `Start-Process -FilePath "notepad.exe"`
-
- `# Stoppa Notepad`
- `Get-Process -Name notepad | Stop-Process`

Quiz:

https://forms.gle/b93SK11nePfjryaG6